Robot-Arm Tennis AI

物理予測・学習・オンライン適応で打ち合う 8 自由度ロボットアーム

Physical AI 応用1 最終課題 ／ OmniCampus アカウント: yoichiozaki ／ 2026 年 10 月

コード: https://github.com/yoichiozaki/physical-ai-robot-sports （robot-arm-tennis/） ／ 動画: YouTube「PAI最終課題_yoichiozaki」

1. 背景と目的

テニスでは、空気抵抗と回転で曲がりバウンドで変化する 3 次元のボールを、0.6–1 秒のうちに予測し、全身（移動と腕）を協調させて「狙った場所へ、狙った速さで」打ち返す必要がある。人は経験（学習）で戦術と感覚を磨き、コートや風が変われば数球で適応する。本課題では、この Physical AI の典型的な難しさを持つ題材として、MuJoCo 上でロボットアーム同士がテニスをする環境を一から構築し、物理モデルと学習の役割分担を検証した。

研究課題: 物理モデルによる予測・計画に、学習（戦術の強化学習、着地誤差の残差学習）とオンライン適応を組み合わせると、対戦性能と未知環境への頑健性は向上するか？

2. タスク概要

ロボット用のミニテニスコート（6.0 m × 2.4 m、ネット高 0.35 m）で、実物の硬式テニスボール（半径 33 mm、57 g）を使う。重力は縮尺できないため、球速 3–7 m/s・飛行 0.6–1.2 秒の「ミニテニス」とした。ルールはシングルス相当: サーブはトスしたボールを 1 バウンド後に打ち、相手コートに入れる。2 バウンド・アウト・ネット・ボディ接触・二度打ちは失点、サーブは 1 ポイントごとに交代し、ポイント間には 1.5 秒の間（両ロボットが構えに戻る）を置く。30 秒で決着しないポイントは打ち切り、最後の打球が相手側でバウンド済みなら打者の得点とする（両者共通）。評価の 1 試合は 20 ポイント。

相手は人間らしい不完全さを持つスクリプト型 5 種（安定型・ランダム・攻撃型・ロブ・低速）で、反応遅延 0.08–0.15 s をわずかに不正確な自分の物理モデルで外挿補償し、狙いには誤差がある。環境条件は 4 種: 標準、ランダム化（学習分布: コート反発 0.68–0.82・摩擦・ラケット反発・抗力 0.75–1.3 倍・風 0.8 m/s 以下・相手速度・センサノイズ）、未知: 強風（横風 2–2.5 m/s + 抗力 1.5–1.7 倍 + ノイズ 2 倍）、未知: コート（よく弾み滑るコート、反発の強いラケット、1.2 倍速い相手、ノイズ 2 倍）。未知条件は学習・残差モデルの学習に一切使っていない。

3. システム構成

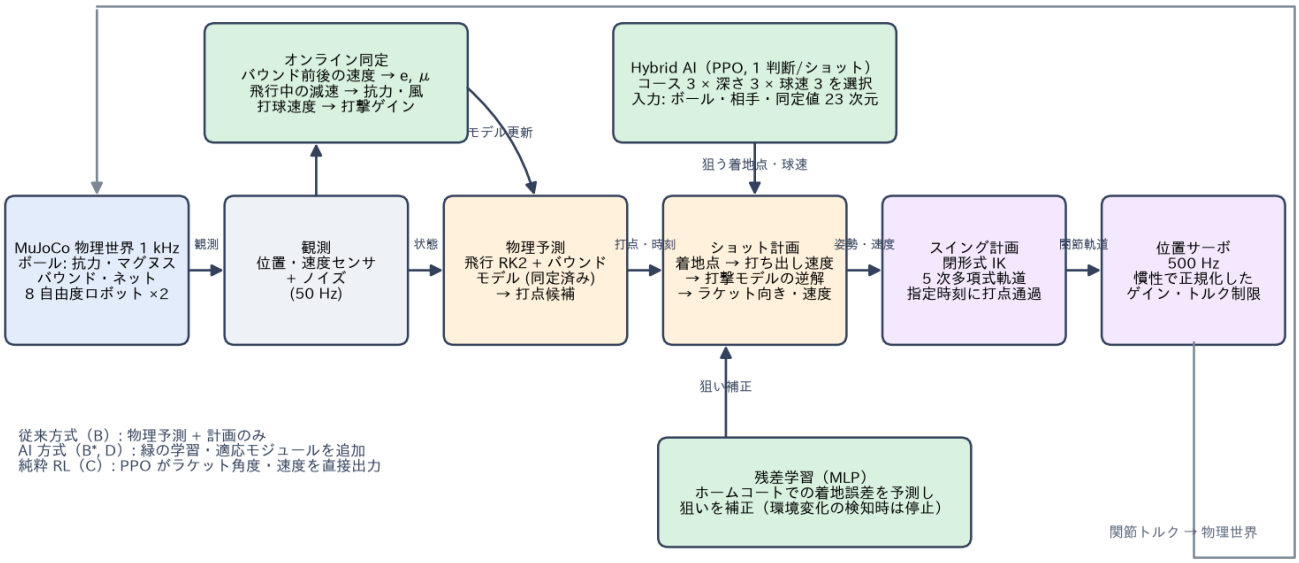


図1 システム構成。物理 1 kHz・サーボ 500 Hz・観測と判断 50 Hz・戦術判断はショットごと（約 1 Hz）。緑が学習・適応モジュール。

シミュレーション（sim）・物理予測と計画（control）・学習（learning）・評価（evaluation）・可視化（viz）を疎結合にし、すべての制御器は最終的に「打点・時刻・ラケットの向きと速度」を経てスイング計画とサーボに渡る。学習は「どこへ打つか」（D）または「ラケットをどう振るか」（C）だけを担い、関節トルクは常に物理的な制約の中で生成される。

4. 物理シミュレーションとロボット

ロボットは、床の 2 軸直動ガントリー（前後 1.5 m・左右 2.8 m）に、肘付き 6 軸アーム（球面手首）とラケット（打球面 30 × 23 cm）を載せた 8 自由度機械で、2 台が向かい合う。アームは閉形式の逆運動学を持ち（MuJoCo の順運動学と

1e-9 以内で一致することをテストで確認）、各関節は慣性から定めたゲインの位置サーボ（速度・加速度フィードフォワード、関節間の慣性結合を考慮したトルク上限、重力補償）で駆動する。モデルは外部アセットを使わずコードから手続き生成した。

ボールの空力は MuJoCo の楕円体流体モデル（抗力 + マグヌス力）で表し、接触は明示的な接触ペアに限定して時定数を調整した（1 kHz で固有振動数 600 rad/s ではバウンドの反発係数が衝突速度で 0.69–0.83 にばらついたため 300 rad/s とした）。予測用の物理モデルはシミュレータから同定した（表1）。

要素	同定値	精度・効果
空気抵抗 k_d	0.02025 /m	予測誤差 RMS 2.1 mm（無視すると 18 cm）
マグヌス k_m	0.00148	RMS 2.6 mm（スピン無視で 10 cm）
コートのバウンド	e = 0.735, μ = 0.284, β = 0.802	e のばらつき σ = 0.025、接線速度 RMS 0.04 m/s
ラケット打撃（実スイング）	a = -0.689, b = 1.541, c_t = 0.972	法線方向 RMS 0.20 m/s（184 打）
遅い球の打撃（サーブ）	b × 1.106（入射 1 m/s 以下）	σ = 0.004（12 本）。サーブの着地誤差 +0.8 m → ±0.2 m

表1 物理モデルの同定（scripts/calibrate.py, calibrate_impact.py）。打撃モデル: v_out = c_t v_in,t + (a v_in,n + b s) n。

5. 物理予測制御（従来方式 B）

観測（50 Hz、位置・速度センサ + ノイズ）から、RK2（5 ms）で空気抵抗とバウンドを含む軌道を 2.2 秒先まで予測し、腕の届く範囲を通る時刻・位置を打点候補として列挙する。各候補で、狙った着地点に落とす打ち出し速度を弾道ソルバ（抗力込みの反復）で求め、打撃モデルの逆解 n = normalize(v_out − c_t v_in)、s = (v_out·n − a v_in·n)/b からラケット面の向き n とラケット速度 s を決める。これを閉形式 IK で関節角に変換し、関節速度は重み付き最小ノルムで配分、準備 → スイング → フォロースルーを区分 5 次多項式で「指定時刻に指定の向きと速度で打点を通過する」軌道として生成する。到達時間・関節速度・トルクの制約を満たす候補のうち最良を選ぶ。

開発中の主な発見: (1) 軌道セグメント連結時に終端速度を 0 扱いしていたバグで打撃誤差が 1.1 m/s あり、修正で 0.20 m/s に改善した。(2) バウンド前にスイングを確定すると、不規則なバウンドで打点が約 10 cm ずれてフレームに当たるため、バウンド直後の打点に罰則を設け、バウンドを観測してから打点を確定する（人の「ボールを見て打つ」に相当）。(3) 初回の評価で、ポイント終了と同時に次のトスを出していたため、大きく振られたロボットが自分のサーブに間に合わず、強化学習と手作りルールの両方がこの規則の穴を利用していた。1.5 秒の間を入れ、学習と評価をすべてやり直した。同時に、サーブ（ほぼ静止した球の打撃）が 0.8 m 長くなる打撃モデルの偏りを見つけ、遅い打撃のゲインを同定して解消した。

6. 学習と適応

6.1 残差学習（教師あり）

ボールマシン練習（ドリル）で、計画した狙いと実際の着地点の差を、打点・速度・ラケット姿勢など 16 特徴から予測する MLP（16→64→64→2）を学習し、狙いを逆方向に補正する。ホームコート（標準条件）の 4,575 球で学習し、検証誤差の中央値は 19.9 → 9.3 cm。条件を毎球ランダムに変えたデータでは誤差が予測不能（ほぼ改善なし）で、残差学習は「同じ環境で繰り返し練習する」ことで効く、という人の練習と同じ性質を示した。

6.2 オンライン同定（適応）

試合中の自分の観測だけから物理モデルを更新する。バウンド前後 3 サンプルの速度から反発係数 e と摩擦 μ を推定（直近 9 回の中央値で外れ値に頑健）、バウンドのない区間の水平減速度から抗力係数と風速を再帰最小二乗で推定（マグヌス力の影響が小さい水平成分のみ使用）、自分の打球の初速から打撃ゲインを推定する。推定値が標準モデルから有意に離れた（変化検知）ときだけ予測・ショット計算に反映し、そのとき残差モデルは停止する（残差は標準モデル用に学習したもので、環境が変わると逆効果になるため）。

6.3 強化学習（PPO）

1 判断 = 自分のショット 1 回とし、相手の打球が自分に向かってきた瞬間（またはサーブ）に方策を呼ぶ Gymnasium 環境を作った。観測は 23 次元（ボール位置・速度、予測バウンド点、自分と相手の位置、相手ラケット、サーブフラグ、オンライン同定値の偏差）。D（Hybrid AI）は MultiDiscrete [3, 3, 3]（左右 × 深さ × 球速 = 27 通りの狙い）を出力し、物理層（予測・ショット解・残差・同定）が実行する。C（純粋 RL）は Box(3)（ラケット面の仰角・方位角・ラケット速度）を直接出力し、打球がどう飛ぶかを自分で学ぶ必要がある（打点への移動のみ物理層を使う。関節レベルの end-to-end RL は接触が疎で予算内に学習不能と判断）。報酬は得点 +1・失点 −1、自分のショットがインで +0.1、1 エピソード = 4 ポイント、相手 5 種 × 標準/ランダム化で学習した。学習量は D 65,536 判断（66 分）、C 49,152 判断（40 分）、いずれもローカル CPU（16 並列環境 × 2）。

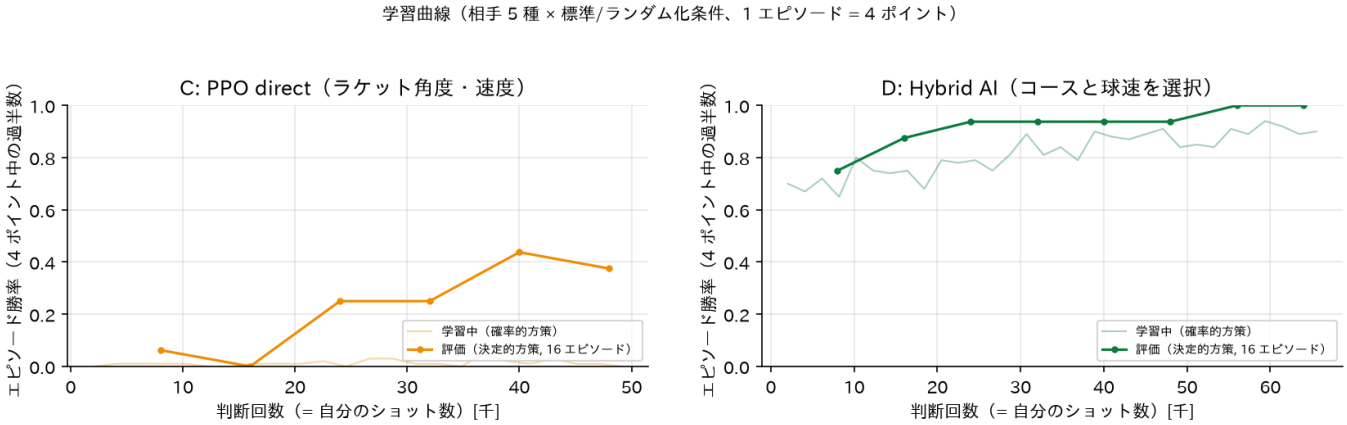


図2 学習曲線（判断回数あたりのエピソード勝率）。評価は決定的方策・16 エピソード。

7. 実験設定

記号	制御器	内容
A	素朴な制御	相手の球を等速直線と仮定（重力・バウンドなし）、固定ストローク。自分のサーブのみ物理を使用
B	物理予測制御	4–5 節の物理層。狙いは常にセンター
B+	物理 + ルール戦術	相手ラケットから遠いコーナーへ（オープンコート）
B*	B+ + 学習・適応	B+ に残差学習（変化検知で停止）とオンライン同定を追加
C	純粋 RL	PPO がラケット角度・速度を直接出力（オンライン同定あり）
D-	Hybrid AI（適応なし）	D の方策を、オンライン同定・残差なしの物理層で実行（アブレーション）
D	Hybrid AI（提案）	PPO が狙いを選び、B* と同じ物理・学習・適応層が実行

表2 比較する制御器。各制御器 × 相手 5 種 × 条件 4 種 × シード 12（20000–20011、学習と重複なし）= 計 1680 試合。同じシードは同じ初期条件・同じ環境パラメータ（対応のある比較）。

8. 結果

制御器	標準	ランダム化	未知: 強風	未知: コート	全体	ラリー長	判断時間
A: Naive	0.9% ±0.5	11.1% ±3.2	14.5% ±2.7	45.6% ±4.4	18.0%	1.6	0.3 ms
B: Physics	58.5% ±4.5	64.9% ±5.6	69.3% ±4.5	67.3% ±4.3	65.0%	8.5	0.7 ms
B+: Physics+Rule	70.8% ±6.2	73.5% ±5.4	66.7% ±3.8	70.8% ±3.4	70.4%	4.1	0.5 ms
B*: Physics+Adapt	75.8% ±6.5	84.9% ±4.5	72.2% ±3.6	76.8% ±2.7	77.4%	4.3	0.7 ms
C: PPO direct	54.4% ±6.0	38.4% ±4.3	44.0% ±4.1	57.3% ±4.2	48.5%	2.6	1.1 ms
D-: Hybrid w/o adapt	91.2% ±3.2	85.1% ±3.3	73.3% ±3.1	45.3% ±3.5	73.8%	1.9	1.6 ms
D: Hybrid AI	91.6% ±3.0	89.5% ±3.3	76.8% ±2.4	71.7% ±2.9	82.4%	2.5	1.9 ms

表3 ポイント取得率（平均 ± 95% 信頼区間）。ラリー長 = 1 ポイントあたりの総打球数。判断時間 = 自ロボットの 1 制御周期あたりの計算時間。

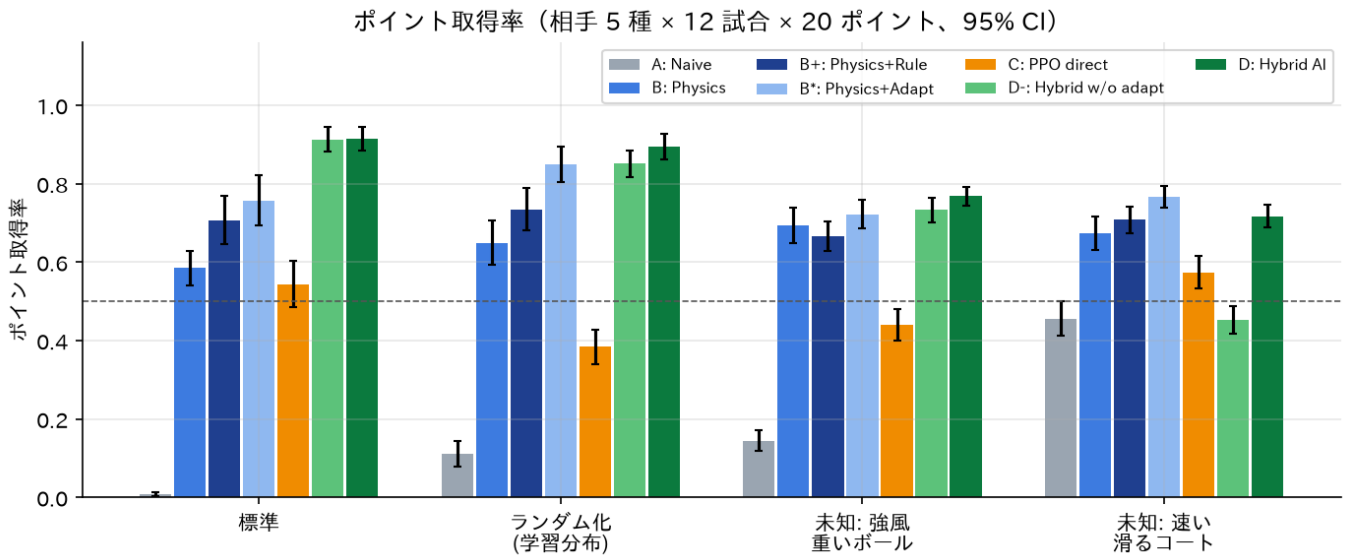


図3 条件別のポイント取得率。破線は 50%（互角）。

比較	条件	ポイント取得率の差（対応のあるブートストラップ）
D – B	全条件	+17.4 pt [95% CI +14.5, +20.3]（有意）
D – B	標準	+33.1 pt [95% CI +28.8, +37.4]（有意）
D – B	未知: 強風	+7.5 pt [95% CI +3.1, +12.2]（有意）
D – B	未知: コート	+4.3 pt [95% CI +0.0, +8.7]（有意）
D – B+	全条件	+12.0 pt [95% CI +9.8, +14.2]（有意）
D – B+	標準	+20.8 pt [95% CI +16.7, +25.1]（有意）
D – B+	未知: 強風	+10.2 pt [95% CI +6.5, +14.0]（有意）
D – B+	未知: コート	+0.8 pt [95% CI -2.9, +4.7]（有意差なし）
D – B*	全条件	+5.0 pt [95% CI +3.0, +7.0]（有意）
D – B*	標準	+15.8 pt [95% CI +11.9, +20.1]（有意）
D – B*	未知: 強風	+4.6 pt [95% CI +1.1, +8.2]（有意）
D – B*	未知: コート	-5.1 pt [95% CI -8.3, -1.8]（有意）
D – C	全条件	+33.9 pt [95% CI +30.6, +37.2]（有意）
D – C	標準	+37.2 pt [95% CI +31.4, +42.9]（有意）
D – C	未知: 強風	+32.8 pt [95% CI +27.8, +38.0]（有意）
D – C	未知: コート	+14.3 pt [95% CI +9.5, +19.2]（有意）
D – D-	全条件	+8.6 pt [95% CI +6.6, +10.6]（有意）
D – D-	標準	+0.3 pt [95% CI -1.4, +2.2]（有意差なし）
D – D-	未知: 強風	+3.5 pt [95% CI +0.5, +6.7]（有意）
D – D-	未知: コート	+26.3 pt [95% CI +22.6, +30.1]（有意）
B* – B+	全条件	+7.0 pt [95% CI +5.2, +8.8]（有意）
B* – B+	標準	+5.0 pt [95% CI +1.7, +8.2]（有意）
B* – B+	未知: 強風	+5.6 pt [95% CI +1.3, +9.8]（有意）
B* – B+	未知: コート	+5.9 pt [95% CI +2.7, +9.4]（有意）
B – A	全条件	+47.0 pt [95% CI +44.0, +50.0]（有意）
B – A	標準	+57.6 pt [95% CI +53.3, +61.8]（有意）
B – A	未知: 強風	+54.8 pt [95% CI +49.8, +59.4]（有意）
B – A	未知: コート	+21.7 pt [95% CI +17.2, +26.3]（有意）

表4 主要な比較。差 = 同じ (相手, 条件, シード) の試合どうしの差の平均。

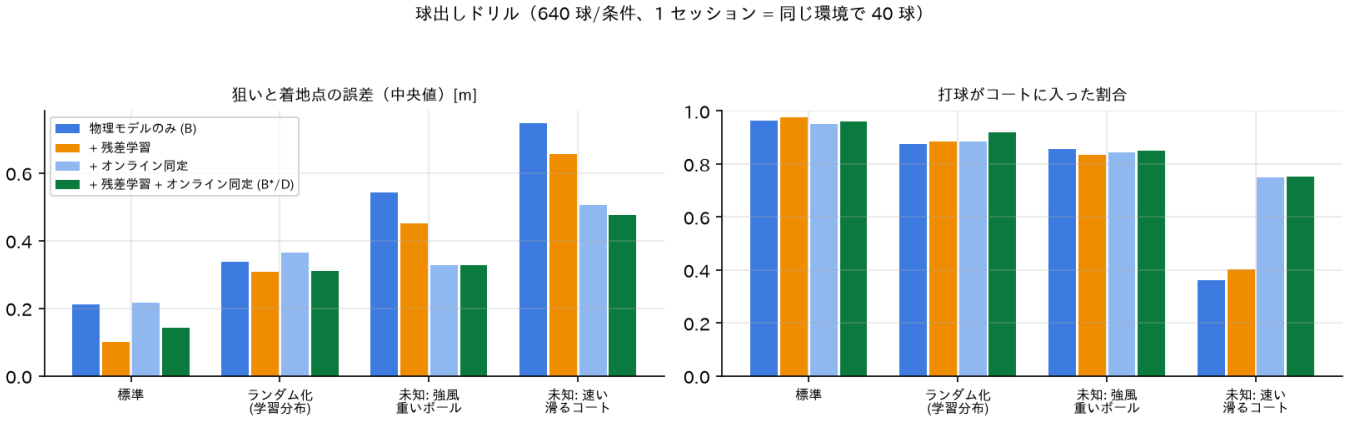


図4 球出しドリルでの打球精度（同じ環境で 40 球のセッション × 16）。残差学習はホームコートで誤差を半減し、オンライン同定は未知のコートで打ち返せる球を 6 倍にした。

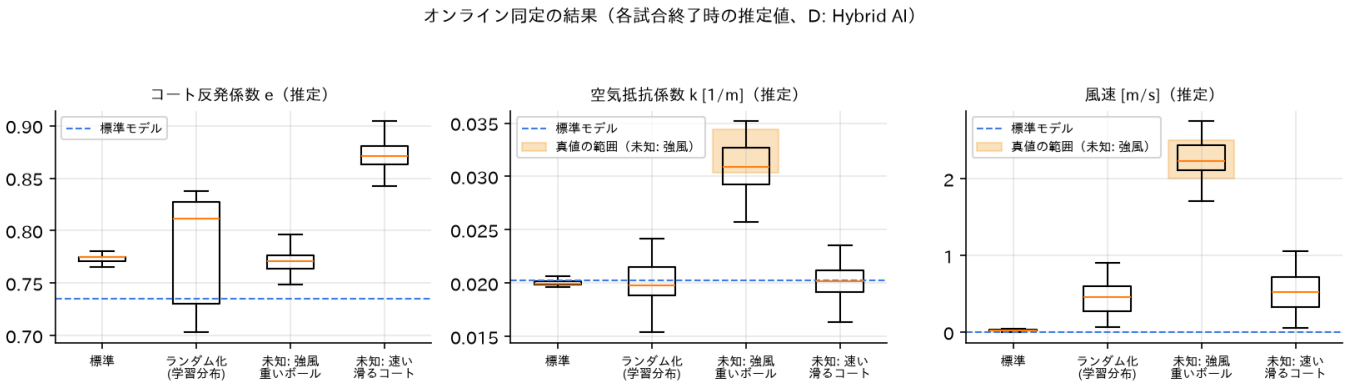


図5 オンライン同定の推定値（各試合終了時）。未知条件では、反発係数・抗力・風を正しい方向・大きさで推定している。

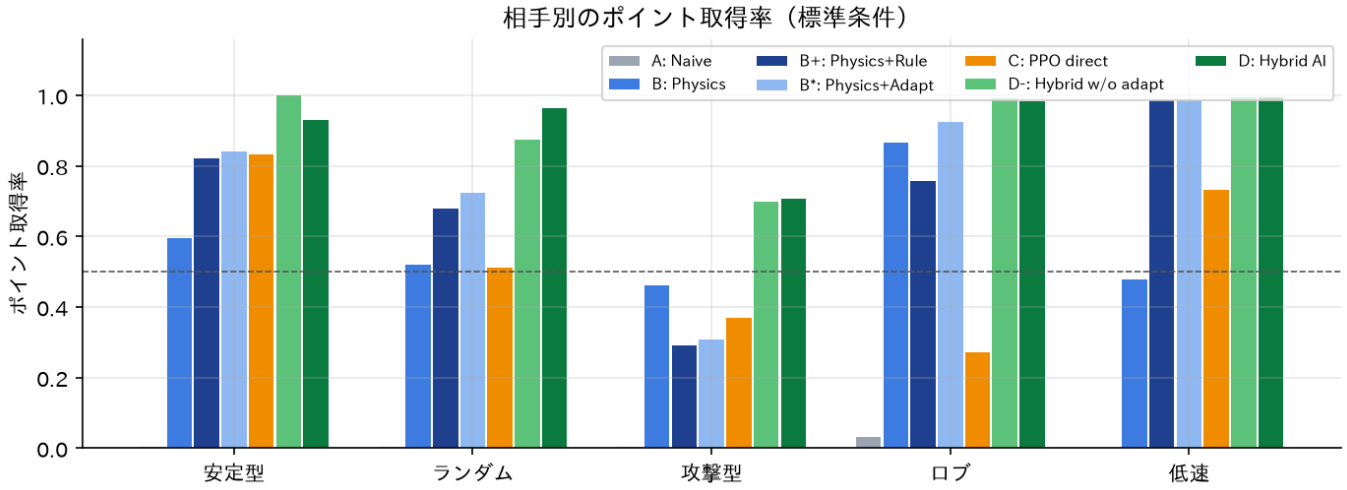


図6 相手別のポイント取得率（標準条件）。

9. 考察

- **物理モデルの効果**: B − A = +47.0 pt [95% CI +44.0, +50.0]（有意）（全条件）。重力・空気抵抗・バウンドを予測できなければ打点に間に合わず、3 次元の打ち合いは物理予測なしには成立しない。学習以前に、物理モデルが性能の土台になっている。
- **戦術の学習**: 同じ物理・学習・適応層の上で、狙いを手作りルールで決める B* と PPO で決める D を比べると、D − B* = +5.0 pt [95% CI +3.0, +7.0]（有意）（全条件; 条件別: 標準 +15.8 pt（有意に高い）、ランダム化 +4.6 pt（有意に高い）、未知: 強風 +4.6 pt（有意に高い）、未知: コート -5.1 pt（有意に低い））。強化学習は相手と状況に応じた狙いを獲得し、手作りの戦術を上回った。差が最も大きいのは攻撃型の相手（標準条件で D 71% 対 B* 31%）。D の選択を集計すると、最速の球（7 m/s）が 100%、センターへの球は 0% で、常に速い球を左右に打ち分け、相手に応じて深さと左右の配分を変えていた（攻撃型の相手には深いボールが 71%）。
- **オンライン適応**: 適応層の有無だけが違う組で、B* − B+ は 標準 +5.0 pt（有意に高い）、ランダム化 +11.4 pt（有意に高い）、未知: 強風 +5.6 pt（有意に高い）、未知: コート +5.9 pt（有意に高い）、D − D− は 標準 +0.3 pt（有意差なし）、ランダム化 +4.4 pt（有意に高い）、未知: 強風 +3.5 pt（有意に高い）、未知: コート +26.3 pt（有意に高い）。ドリルでは未知のコートで打ち返せた球が 72 → 415 球（640 球中）、イン率 36% → 75% に改善した。未知の環境での頑健性は、学習した戦術ではなく、物理モデルを観測から更新する仕組みが担っている。
- **純粋 RL との比較**: D − C = +33.9 pt [95% CI +30.6, +37.2]（有意）（全条件）。ラケット角度と速度から打球の行き先を学ぶ C は、同じ程度の学習量では打球を安定してコートに入れられない（イン率 57–79%、D は 71–97%）。打撃モデルの逆解で「狙った場所へ打つ」を物理層が保証する D は、27 通りの狙いを選ぶだけでよく、学習の負担が桁違いに小さい。
- **残差学習の位置づけ**: ホームコートでは着地誤差の中央値を 21 → 10 cm に減らしたが、環境が変わると補正が外れて逆効果になりうる（同定でモデルを変えたまま併用すると標準条件でも誤差が 3 倍）。変化検知で残差を止める設計により、ホームでの精度と未知環境での頑健性を両立させた。
- **研究課題への答え**: 物理モデルを土台に、学習は「どこへ打つか」（戦術）と「モデルのずれ」（残差）を、オンライン同定は「環境の変化」を担うように役割を分けることで、学習分布内の対戦性能は大きく向上し（D は全条件で最高またはそれに近いポイント取得率）、未知の環境でも性能の崩れを大幅に抑えられた。学習だけ（C）では打球を制御できず、物理だけ（B）では戦術が単調になる。ただし、未知のコートでは学習した戦術が手作りルールに劣った（D − B* = -5.1 pt [95% CI -8.3, -1.8]（有意））。方策は同定値を観測として受け取るが、学習中に見ていない範囲の環境では狙い

の選び方まで適応できておらず、戦術そのものの適応（未知環境を含む学習分布、メタ学習など）は今後の課題である。

10. 発展 1: 自己対戦でさらに強くなれるか

D を出発点に、自分の過去の世代と戦い続ける自己対戦リーグ（AlphaStar 方式の簡略版。1.2 万判断ごとに世代を追加し、学習者が苦手な過去の世代を優先して選び、2 割はスクリプト相手）を CPU で 4 時間（9.7 万判断）学習した。全世代・B 系・スクリプト相手の総当たり戦（544 試合）から Elo を推定すると、全世代が 1484–1507 の範囲で、強さは変わらなかった。

原因はサーブだった。学習済みの方策がサーブすると、相手が誰でも 99.9% がエースになる（他の選手のサーブは 7.3%）。「深く・受け手のフォア側・7 m/s」のサーブがベースライン際で低く弾み、受け手ロボットの台座の横を抜ける。レールは取り付け位置より後ろへ下がれず、腕も台座に近すぎる球は打てないため、受け手は打点を見つけられない（設計上の死角）。D 自身が元の学習でこの死角を見つけており、自己対戦は世代 1 以降この 1 本に収束した。サーブで勝負が決まるゲームでは、強さの差が生まれない。

そこで実際のテニスと同じサービスボックス（サーブはネットから 1.62 m 以内、実コートと同じ比率）を加え、D を微調整してから自己対戦をやり直した（7.2 万判断）。学習済み同士のラリーは 1 ポイント平均 3.2 打になり、Elo は +215 まで上がって頭打ちになった（図 7）。ただし上昇の大部分は出発点（世代 0）への圧勝（最終世代が世代 0 から 98% のポイント）によるもので、固定の相手へのポイント取得率はほぼ変わらない（対 B 58% → 67%、対 B* 92% → 88%）。公平なルールでの自己対戦は、自分の系統に勝つ戦術（深く速い球）を素早く見つけるが、汎用的な強さにはつながりにくかった。また強化学習は、ルールや設計の穴（ポイント間の間、届かないサーブ）を真っ先に突く「デバッガ」としても働いた。

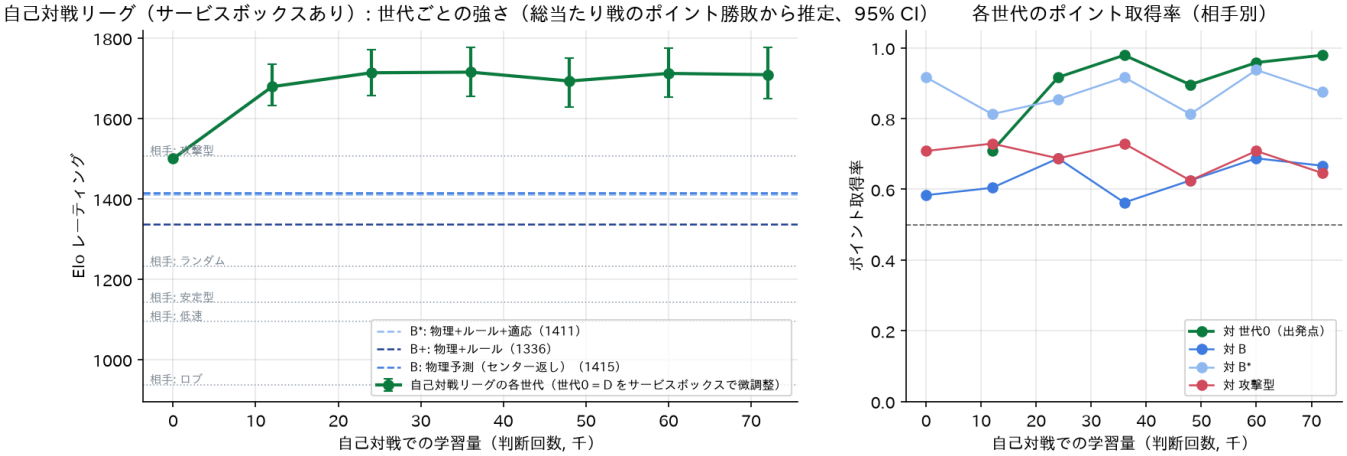


図7 サービスボックスありの自己対戦。左: 世代ごとの Elo（95% CI）。右: 世代ごとのポイント取得率（相手別）。

11. 発展 2: VLA はテニスができるか（惑星テニス）

VLA（Vision-Language-Action）モデルの SmolVLA（4.5 億パラメータ、LeRobot 0.4.4）を、物理エキスパートの成功デモ 1,673 本（26 万フレーム。カメラ画像 2 枚＋関節状態＋英語の指示文 → 20 ms 後の関節目標と狙い）で微調整した（Colab の A100、2 万ステップ、3.9 時間）。課題は言語指示つきの返球ドリルで、ゾーンを指定する指示 6 種と、相手を見ないと従えない指示 2 種（“away from / straight at the opponent”）がある。重力だけを変えると軌道の形はそのままで時間だけが伸びる（火星 1.6 倍、月 2.5 倍遅い。テストで確認）ことを使い、地球・火星・月で「試合の速さ」だけを変えた。推論の遅れはシミュレーション時間で再現した（A100 の実測 348 ms）。

制御器	地球 命中 / 指示どおり	火星 命中 / 指示どおり	月 命中 / 指示どおり
物理エキスパート（教師）	98% / 89%	98% / 96%	100% / 100%
VLA 単独・遅れ 0	4% / 0%	5% / 0%	7% / 0%
VLA 単独・実測の遅れ 348 ms	0% / 0%	2% / 0%	4% / 0%
VLA 単独＋ボール位置入力・遅れ 0	49% / 4%	47% / 3%	37% / 2%
VLA 単独＋ボール位置入力・348 ms	59% / 3%	50% / 3%	55% / 3%
階層型・実測の遅れ 348 ms	98% / 89%	98% / 93%	100% / 100%

表5 VLA の評価（各条件 100 球、学習と別のシード）。命中 = ラケットに当たった割合、指示どおり = 指示したゾーンに着地した割合。

VLA 単独（画像 → 関節）は、考える時間をゼロにしても、2.5 倍遅い月面でもほとんど当たらない（月で命中 7%）。遅れが原因ではなく、そもそもボールの近くに振れていない（外れたときのラケットとボールの最接近距離の中央値は 64 cm、エキスパートの打球時は 5–9 cm）。画像の認識を不要にする対照実験として、ボールの位置と速度（真値）を入力に加えて同じ設定で学習し直すと、命中は遅れ 0 で地球 49%・火星 47%・月 37% に上がり、外したときの最接近距離も約 25 cm に縮んだ。画像から速いボールを捉えることが最初のボトルネックである。それでも当たった球のうちコートに入るの

は 21%、そのうち指示どおりのゾーンは 28%（1 ゾーンの面積はコートの約 1/6）で、打球の向きと速さを決める精度が次の壁になる。遅れの影響は単調ではない。同じ切り替え周期で比べた遅れそのものの影響は 10 ポイント以内だったが、0.1 秒ごとに 0.1 秒古い答えへ切り替えると火星 7%・月 10% に崩れ、0.36 秒ごとなら 50–69% だった（図 10）。end-to-end では、遅れよりも行動チャンクのつなぎ方が成否を分ける。

一方、VLA に狙いだけを決めさせ、物理層が打つ階層型は、どの世界でも指示どおりのゾーンに平均 94% 返した（相手を見ないと従えない指示でも 94%）。答えが遅すぎると打点の確定に間に合わず指示に従えなくなるが、その限界（成功率が半分になる遅れ）は地球 1.1 秒・火星 1.8 秒・月 3.1 秒で、世界の時間の伸びにほぼ比例した（遅れ × 速さで曲線が重なる、図 9）。汎用モデルは「どこへ打つか」を画像と言葉から決める脳としては有能だが、10 ms 単位のタイミングが要る動きは物理モデルに任せるべき、というのが結論である。

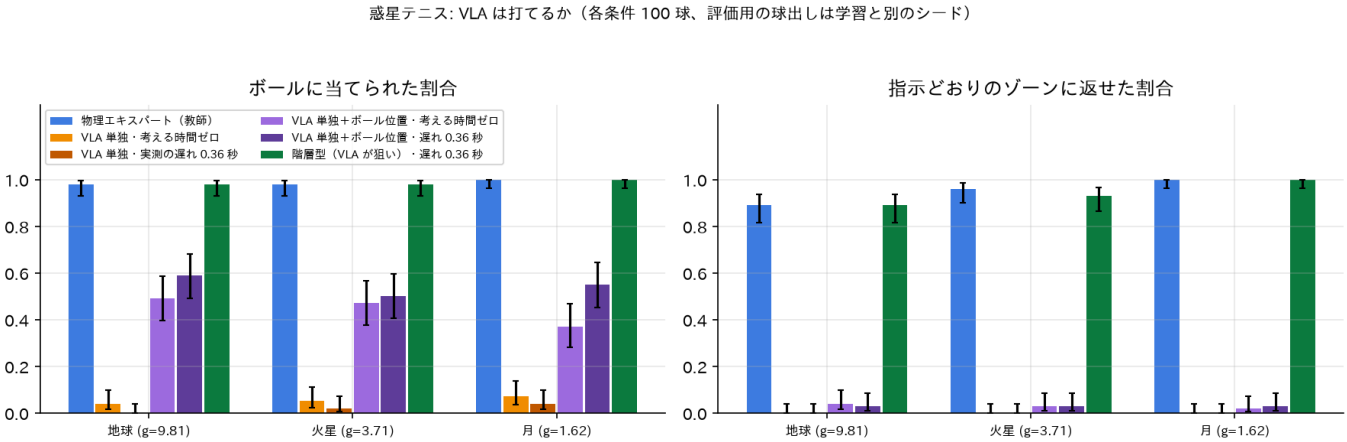


図8 惑星テニス: 各制御器の命中率と指示どおりの割合（95% CI）。

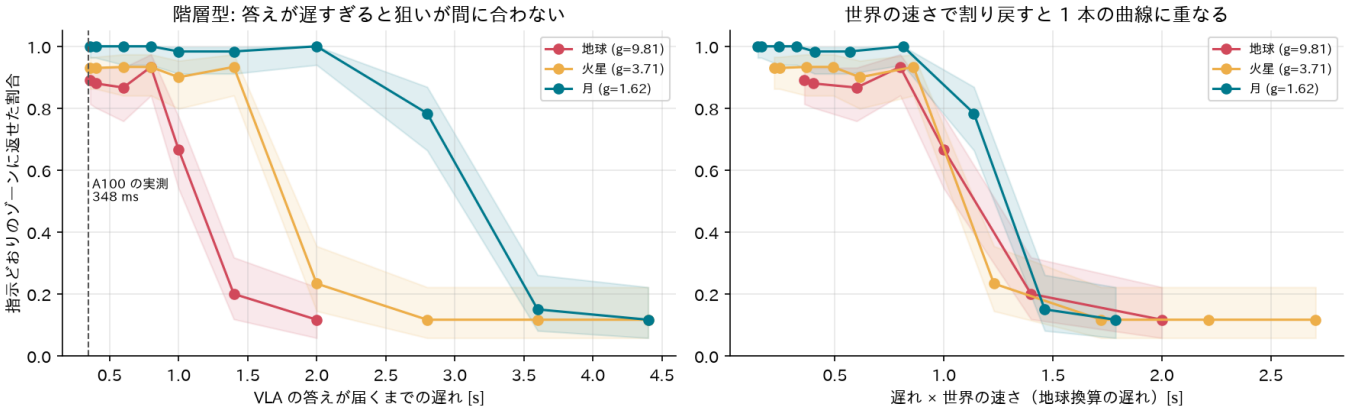


図9 階層型: VLA の答えの遅れと指示どおりの割合。右は遅れに世界の速さを掛けて地球換算したもの。

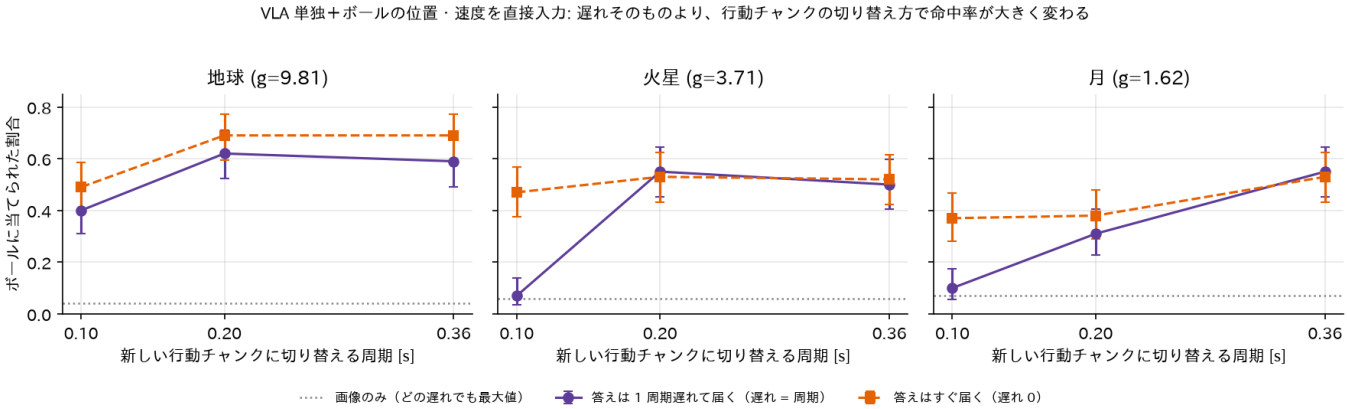


図10 対照実験: ボールの位置と速度を直接入力した VLA 単独の命中率（各 100 球、95% CI）。横軸は新しい行動チャンクに切り替える周期。点線は画像のみの VLA の最大値。

12. 限界と今後

- シミュレーションのみ（実機・実カメラは未検証）。センサは位置・速度にガウスノイズを加えたもので、スピンは観測しない。
- 相手はスクリプト型 5 種と自己対戦の過去の世代。人間の打球データとの対戦は今後の課題。

- ロボットはレールの取り付け位置より後ろへ下がれない（サーブの死角）。サービスボックスで回避したが、本来は設計の改善が必要。
- 強化学習の予算は CPU で数時間規模。方策は 27 通りの離散的な狙いで、連続的な狙い・回転の選択は未対応。
- VLA は SmolVLA 1 種・デモ 1,673 本・2 万ステップのみ。より大きなモデル、手首カメラ、推論の高速化、行動チャンクのなめらかなつなぎ方（temporal ensembling、real-time chunking）は未検証。
- オンライン同定は観測が十分にある量（反発・摩擦・抗力・風・打撃ゲイン）に限った。スピン量や突風のような時間変化は扱っていない。
- 未知条件のドリル結果は適応手法の開発中に参照した（試合評価のシードは開発に使っていない）。

13. 再現方法

Dev Container 内で robot-arm-tennis/ に移動し、python -m pytest（テスト）、python -m tennis.learning.train --config configs/training/ppo_hybrid.yaml（学習）、python scripts/evaluate.py --config configs/evaluation/main.yaml（評価）、python scripts/make_figures.py、python scripts/make_video.py、python scripts/make_report.py（図・動画・本資料）。乱数はすべてシード指定で、動画のクリップは評価と同じコードの決定的な再生である。自己対戦は configs/training/ppo_league*.yaml とscripts/league_eval.py、VLA は scripts/vla/（Colab で make_dataset → colab/train.sh → colab/eval_all.sh。対照実験は make_dataset --ball-state と colab/eval_ball.sh）。詳細は README.md と EXPERIMENTS.md。